

FINAL (ATTRIBUTES/PARAMETERS)

Variable	Method	Class
Constant	No overriding	No inheritance

INITIALISATION

1) Default-Values ↓
2) Attribute Assignments
3) Initialisation block
4) Constructor

DEFAULT VALUES

Type	Default	Type	Default
boolean	false	char	'\u0000'
byte	0	short	0
int	0	long	0L
float	0.0f	double	0.0d

TYPES

Type	Size (bit)	From	To
byte	8	−128	127
short	16	−32'768	32'767
char	16	UTF-16 chars	
int	32	−2 ³¹	2 ³¹ − 1
long	64	−2 ⁶³	2 ⁶³ − 1
float	32	±1.4 · 10 ^{−45}	±3.4 · 10 ³⁸
double	64	±4.9 · 10 ^{−324}	±1.7 · 10 ³⁰⁸

// short * int ⇒ int
// float + int ⇒ float
// int / double ⇒ double
// int + long * float ⇒ float
// 0.0 / 0.f ⇒ NaN

long l = 1L;
long ll = 0b1L;
float f = 0.0f;
double d = 0.0d;

12 = '.'; // implicit int/char conversion
0.1 + 0.1 ≠ 0.2; // true
5/2 = 2; // true, int div truncates to 0
NaN = NaN; // false
Integer.MAX_VALUE + 1 = Integer.MIN_VALUE;
1 / 0.0 = Double.POSITIVE_INFINITY; // true

var ints = new ArrayList<Integer>();
int[] jnts = new int[69];
int[][] matrix = new int[6][9];

if (obj instanceof ArrayList<Integer>)
(ArrayList<Integer>)obj).add(2);

public List<String> method(
BiFunction<Integer, String, List<String>> fn) {
return fn.apply(5, "FooBar");
}

IMPLICIT CASTING

No information loss int→float, to larger type int→long
Sub→Super is implicit, Super→Sub ClassCastException

// explicit casting
float f = (float) 1;
// type conversion
Integer.parseInt("2");
Float.parseFloat("2.0");

WIDENING PRIMITIVE CONVERSION

– If either operand is double, the other is converted to double
– Otherwise, if either is float, the other is converted to float
– Otherwise, if either is long, the other is converted to long
– Otherwise, both operands are converted to type int

VARIABLE ARGS

```
static int sum(int... numbers) {  
int sum = 0;  
for (int i = 0; i < numbers.length; i++)  
sum += numbers[i];  
return sum;  
}
```

MISC

int[] intarr = new int[] {1, 2, 3, 4, 5};
int[] sub = Arrays.copyOfRange(intarr, 1, 3); // 2,3

var intlist = new ArrayList<Integer>();
intarr.length;
intlist.size();

// Multiply first to not lose precision
int percent = (int)((filled * 100) / capacity);

obj.clone();

Math.min(x, y);
Math.max(x, y);

Rekapitulation: Primitive Datentypen
Konversionen

VISIBILITY

public	all classes
protected	package + sub-classes
private	only self
(none)	all classes in same package

PACKAGES

p1.sub won't be automatically imported in p1.
Package name collisions: first gets imported.

package p1;
public class A { }

package p2;
import p1.sub.E;
public class C {
E e = new E();
}

package p1.sub;
public class E { }

package p1; // public class A { }
package p2; // public class A { }

// OK
import p1.A;
import p2.*;

// reference to A is ambiguous
import p1.*;
import p2.*;

// sin, PI
import static java.lang.Math.*;

MODULES

// ./foo/module-info.java
module foo.bar.baz {
exports com.my.package.foo;
}

// ./main/module-info.java
module main.module {
requires com.my.package.foo;
}

ENUMS

public enum Weekday {
MONDAY(true),
TUESDAY(true),
WEDNESDAY(true),
THURSDAY(true),
FRIDAY(true),
SATURDAY(false),
SUNDAY(false);
}

private boolean workDay;

Weekday(boolean workDay) { // private constructor
this.workDay = workDay;
}

public boolean isWorkDay() {
return workDay;
}

switch (wd) {
case Weekday.MONDAY:
case Weekday.TUESDAY:
System.out.println("First two");
break;
default:
System.out.println("Rest");
}

public enum Level {
LOW,
MEDIUM,
HIGH
}

SWITCH

switch (x) {
case 'a':
System.out.println("1");
break;
default:
System.out.println("2");
}

int v = switch (x) {
case 'a' -> 1;
default -> 2;
}

SHADOWING

Variables with same names in same scope

HIDING

Variables with same names in different classes
Gets statically chosen by compiler

super.description = ((Vehicle)this).description
super.super // doesn't exist, use v
((SuperSuperClass)this).variable
// for multiple parent interfaces v
ParentInterface1.super.defaultMethod();
// ^ executes defaultMethod on ParentInterface1
ParentInterface2.super.defaultMethod();

OVERLOADING

Methods with same names but different parameters
Gets statically chosen by compiler

void print(int i, double j) { } // 1
void print(double i, int j) { } // 2
void print(double i, double j) { } // 3

print(1.0, 2.0); // 3
print(1,2); // error: reference to print is ambiguous
print(1.0, 2); // 2
print(2.0, (double) 2); // 3

OVERRIDING

Methods with same names and signatures
Dynamically chosen (Dynamic dispatch / Virtual call)
Error: Cannot override the final method...
Error: Cannot be subclass of final class...

class Fruit {
void eat(Fruit f) { System.out.println("1"); }
}

class Apple extends Fruit {
void eat(Fruit f) { System.out.println("2"); }
void eat(Apple a) { System.out.println("3"); }
}

Apple a = new Apple();
Fruit fa = new Apple();
Fruit f = new Fruit();

a.eat(fa); // 2
a.eat(a); // 3
fa.eat(a); // 2
fa.eat(fa); // 2
f.eat(fa); // 1
f.eat(a); // 1
((Fruit) a).eat(fa); // 3
((Apple) fa).eat(a); // 2
((Apple) f).eat(a); // ClassCastException

ABSTRACT CLASSES

public abstract class Vehicle {
private int speed;
public abstract void drive();
public void accelerate(int acc) {
this.speed += acc;
}
}

public class Car extends Vehicle {
@Override
public void drive() { }
@Override
public void accelerate (int acc) { }
}

a.eat(fa); // 2
a.eat(a); // 3
fa.eat(a); // 2
fa.eat(fa); // 2
f.eat(fa); // 1
f.eat(a); // 1
((Fruit) a).eat(fa); // 3
((Apple) fa).eat(a); // 2
((Apple) f).eat(a); // ClassCastException

INTERFACES

Cannot have Attributes

interface RoadV {
int MAX_SPEED = 120;
void drive();
}

interface WaterV {
int MAX_SPEED = 80;
void drive();
}

class AmphibianMobile implements RoadV, WaterV {
@Override // because ambiguous
public void drive() {
println(RoadV.MAX_SPEED); // MAX_SPEED ambiguous
}
}

interface RoadV { String getModel(); }
interface WaterV { int getModel(); }
// Error, because of different return types
class AmphibianMobile implements RoadV, WaterV { }

INTERFACES DEFAULT METHODS

interface Vehicle {
default void printModel() {
System.out.println("Undefined vehicle model");
}
}

interface RoadV { String getModel(); }
interface WaterV { int getModel(); }
// Error, because of different return types
class AmphibianMobile implements RoadV, WaterV { }

ANONYMOUS CLASSES

var v = new RoadV() {
@Override
public void drive() {
System.out.println("Anon");
}
}

INHERITANCE

public class Vehicle {
private int speed;
public Vehicle(int speed) {
this.speed = speed;
}
}

public class Car extends Vehicle {
private int doors;
public Car(int speed, int doors) {
super(speed);
this.doors = doors;
}
}

Car c = new Car(); // Points to Car
Vehicle v = new Car(); // Points to Car
Object o = new Car(); // Points to Car
// ^static ^dynamic
Car c = (Car) new Vehicle(); // ClassCastException

MORE INHERITANCE

public class Qwer {
public void print() {
System.out.println("1");
}
}

public class Asdf extends Qwer {
@Override
public void print() {
System.out.println("2");
}
public void dostuff () { }

var x = new Asdf();
x.print(); // 2
(((Qwer) x).print(); // 2
(((Qwer) x).dostuff(); // cannot find symbol

Static Type: According to var declaration at compiletime
Dynamic Type: Type of the instance at runtime

00P1 | HS25

Page 1

